Point of Sale System — New Additions and Rationale

Overview

The team's original draft had 9 entities. This document lists everything added beyond that draft, and why: 14 new entities, changes to 6 of the original entities, fixes to 3 issues in the original draft, and the relationships and constraints those additions require. Two of the additions respond directly to instructor feedback: sale events and a customer rewards program. The rest fill gaps between the mini-world's own description and what the original draft's entities could actually support.

New Entities

Filling gaps the original draft left open

Entity	Attributes	Why it was added
CUSTOMER_PHONE	Customer_id, Phone_number	The draft wrote Phone_number in braces, {Phone_number}, meaning a customer can have more than one. A single column on CUSTOMER cannot hold more than one value, so it needed its own table.
SUPPLIER_ORDER	Order_id, Supplier_id, Employee_id, Order_date, Received_date, Status	The relationship list says employees place orders with suppliers, M:N. An M:N relationship needs a table of its own; it cannot be represented with a foreign key on either SUPPLIER or EMPLOYEE alone.
SUPPLIER_ORDER_ITEM	Order_id, Product_id, Quantity, Unit_cost	One supplier order can request more than one product, so each product on the order needs its own row rather than repeating columns on SUPPLIER_ORDER.
SUPPLIER_PRODUCT	Supplier_id, Product_id, Supplier_sku, Unit_cost, Lead_time_days	The original PRODUCT table had one Supplier_id, meaning each product could only come from one supplier. In practice more than one supplier can carry the same product, which needs a table between SUPPLIER and

Entity	Attributes	Why it was added
		PRODUCT.

Supporting operations the description names but does not model

Entity	Attributes	Why it was added
PAYMENT	Payment_id, Transaction_id, Method, Amount, Card_last4, Change_given, Paid_at	The description says the system records "payment information for each transaction," but the draft only had one Payment_method column on TRANSACTION. That cannot record a split payment or the amount actually tendered.
LOYALTY_TRANSACTION	Loyalty_txn_id, Customer_id, Transaction_id, Program_id, Points_change, Reason, Created_at	CUSTOMER only stored a single running Loyalty_points total. That total cannot be checked or corrected without a history of how it was earned and spent.
INVENTORY_ADJUSTMENT	Adjustment_id, Product_id, Employee_id, Quantity_change, Reason, Adjusted_at	The description says managers are the only ones who "update inventory, edit inventory," but nothing recorded who changed a stock count or when.
PRICE_CHANGE	Price_change_id, Product_id, Employee_id, Old_price, New_price, Changed_at	Same issue for the description's "change prices": PRODUCT only stored the current price, with no record of who changed it or what it was before.
REGISTER	Register_id, Register_number, Is_active	The description says employees use a "Virtual Register" to check customers out, but no entity represented a register, so a transaction could not be tied to the one it happened on.
LOW_STOCK_ALERT	Alert_id, Product_id, Triggered_at, Resolved_at, Order_id	PRODUCT already has a Reorder_threshold, but nothing acted on it. This entity records when a product actually crossed that threshold and which

Entity	Attributes	Why it was added
		supplier order resolved it.
TAX_RATE	Tax_rate_id, Category_id, Rate, Effective_from, Effective_to	TRANSACTION records a Tax_amount, but nothing defines the rate that produced it. Rates can differ by category and change over time, so they need their own table rather than a fixed number.

Requested by Uma

Entity	Attributes	Why it was added
SALE_EVENT	Sale_event_id, Name, Description, Start_date, End_date, Is_active	The instructor's feedback said the schema needs to represent sale events. DISCOUNT covers a single price reduction, but nothing represented the named, dated event, such as a Black Friday sale, that several discounts can belong to.
REWARDS_PROGRAM	Program_id, Name, Points_per_dollar, Redemption_rate, Effective_from, Effective_to	The instructor's feedback also asked for a customer rewards program. LOYALTY_TRANSACTION (above) records individual point changes, but nothing defined the rules, such as the earn rate, that produced them, or let those rules change over time without rewriting past history.
REWARD_TIER	Tier_id, Program_id, Name, Min_points, Perk_description	A rewards program typically has tiers, such as Bronze, Silver, and Gold, each with its own perks. This gives customers a status to display and work toward, beyond a raw point total.

Changes to the Original Entities

Entity	Attribute	Why

Entity	Attribute	Why
EMPLOYEE	+ Password_hash	The description says employees must log in before using the system, but the draft had no way to check a password.
EMPLOYEE	+ Supervisor_id (FK => EMPLOYEE, optional)	The relationship list says managers supervise employees, 1:N, but no column recorded who supervises whom.
CUSTOMER	− Phone_number	Moved to the new CUSTOMER_PHONE table, since a customer can have more than one phone number.
CUSTOMER	+ Referred_by_id (FK => CUSTOMER, optional)	The relationship list says customers refer customers, 1:N, but no column recorded who referred whom.
CUSTOMER	+ Tier_id (FK => REWARD_TIER, optional)	Needed once REWARD_TIER exists, so a customer's current tier can be looked up directly instead of recalculated from their full points history every time.
PRODUCT	− Supplier_id	Replaced by the new SUPPLIER_PRODUCT table, since more than one supplier can carry the same product.
PRODUCT	+ Category_id (FK => CATEGORY)	The description says every product belongs to exactly one category, but the draft's PRODUCT table had no column pointing to CATEGORY.
CATEGORY	+ Discount_id (FK => DISCOUNT, optional)	The relationship list says discounts apply to categories, 1:N, but no column carried that link.
DISCOUNT	+ Code	Coupons and promotion codes need to be typed in at the register; Type alone does not

Entity	Attribute	Why
		hold the code itself.
DISCOUNT	+ Amount_type (percent, fixed)	Amount alone does not say whether it means 10% off or $10 off.
DISCOUNT	+ Sale_event_id (FK => SALE_EVENT, optional)	Needed once SALE_EVENT exists, so a discount can optionally belong to a named sale.
TRANSACTION	− Payment_method	Moved to the new PAYMENT table, since a transaction can be paid through more than one method.
TRANSACTION	+ Register_id (FK => REGISTER)	Needed once REGISTER exists, so a transaction can be tied to the register it happened on.
TRANSACTION	+ Status (open, completed, voided)	The description says items can be added to or deleted from a transaction at the register, which only makes sense while the transaction is still open.
TRANSACTION_ITEM	+ Discount_id (FK => DISCOUNT, optional)	Line_total needs to show which discount, if any, reduced that line, and the draft had no way to record it.
TRANSACTION_ITEM	+ Discount_amount	Keeps the amount actually taken off at the time of sale, even if the discount is edited or removed afterward.
REVIEW	+ Customer_id (FK => CUSTOMER)	The description says customers make reviews, but the draft's REVIEW table did not say whose review it was.
REVIEW	+ Transaction_id (FK => TRANSACTION, optional)	A review may be about a specific transaction, not only a product; Product_id already existed.

Entity	Attribute	Why
REVIEW	− Avg_rating	This is a value calculated from all of a product's ratings, not a fact about one review. Storing it on every review risked it going out of sync with the ratings it was supposed to summarize.

Fixes to the Original Draft

- **Role names.** The description names the two employee roles Manager and Cashier, but the constraints list names them Manager and Staff. The combined document uses Cashier, matching the description. The team should confirm which one is correct and use it consistently in the actual database and code.
- **Transaction date.** The description says the system records "date, time" for each transaction, so the Date column should be a full date-and-time value, not a date alone.
- **Spelling.** Costs_price is corrected to Cost_price, and "Sate" is corrected to "State" in both address groups, on CUSTOMER and SUPPLIER.

New Relationships

Relationship	Why
EMPLOYEEs supervise EMPLOYEEs — 1:N	Implements the Supervisor_id column added to EMPLOYEE, and the relationship the original list already named but did not model.
CUSTOMERs refer CUSTOMERs — 1:N	Implements the Referred_by_id column added to CUSTOMER, for the same reason.
CUSTOMERs have CUSTOMER_PHONEs — 1:N	Follows directly from adding the CUSTOMER_PHONE table.
SUPPLIERs supply PRODUCTs — M:N, via SUPPLIER_PRODUCT	Replaces the original 1:N version of this relationship now that a product can have more than one supplier.
EMPLOYEEs place SUPPLIER_ORDERs — 1:N, and SUPPLIERs receive SUPPLIER_ORDERs — 1:N	Together these implement "employees place orders with suppliers, M:N" from the original list, which needs a table in the middle rather than a direct M:N link.
SUPPLIER_ORDERs contain SUPPLIER_ORDER_ITEMs — 1:N, and PRODUCTs appear in SUPPLIER_ORDER_ITEMs — 1:N	Follows from adding SUPPLIER_ORDER_ITEM, so one order can request more than one product.

Relationship	Why
DISCOUNTs apply to TRANSACTION_ITEMs — 1:N	Follows from adding Discount_id to TRANSACTION_ITEM, so Line_total can reflect a discount.
TRANSACTIONs may be the subject of REVIEWs — 1:N	Follows from adding Transaction_id to REVIEW.
TRANSACTIONs are paid through PAYMENTs — 1:N	Follows from splitting Payment_method out into its own PAYMENT table.
REGISTERs process TRANSACTIONs — 1:N	Follows from adding the REGISTER entity for the description's Virtual Register.
PRODUCTs trigger LOW_STOCK_ALERTs — 1:N, and SUPPLIER_ORDERs resolve LOW_STOCK_ALERTs — 1:N	Follows from adding LOW_STOCK_ALERT, so a product's reorder threshold actually leads to a tracked action.
EMPLOYEEs make INVENTORY_ADJUSTMENTs — 1:N, and PRODUCTs have INVENTORY_ADJUSTMENTs — 1:N	Follows from adding INVENTORY_ADJUSTMENT, so stock changes can be traced to who made them.
EMPLOYEEs make PRICE_CHANGEs — 1:N, and PRODUCTs have PRICE_CHANGEs — 1:N	Follows from adding PRICE_CHANGE, for the same reason.
CATEGORYs have TAX_RATEs — 1:N	Follows from adding TAX_RATE, so tax can differ by category.
CUSTOMERs earn LOYALTY_TRANSACTIONs — 1:N, and TRANSACTIONs generate LOYALTY_TRANSACTIONs — 1:N	Follows from replacing a single Loyalty_points total with a full history.
REWARDS_PROGRAMs govern LOYALTY_TRANSACTIONs — 1:N	Ties each point change to the ruleset that produced it, per the instructor's feedback.
REWARDS_PROGRAMs define REWARD_TIERs — 1:N, and REWARD_TIERs classify CUSTOMERs — 1:N	Follows from adding the tier structure to the rewards program, per the instructor's feedback.
SALE_EVENTs group DISCOUNTs — 1:N	Follows from adding SALE_EVENT, per the instructor's feedback, so several discounts can belong to one named sale.

New Constraints

Constraint	Why
An employee's supervisor must hold the Manager	Prevents a Cashier from being listed as

Constraint	Why
role and cannot be the employee themselves.	someone's supervisor, and prevents an employee from supervising themselves.
A customer cannot refer themselves.	Same reasoning, applied to the new Referred_by_id column.
Items can be added to or removed from a transaction only while its status is open.	Matches the description's Virtual Register behavior, and stops a completed sale from being silently edited afterward.
Selling a product decreases its stock only once the transaction's status is completed, and a voided transaction leaves stock unchanged.	Without this, stock would drop the moment an item is added to an open transaction, even if the sale is later cancelled.
Marking a supplier order received increases each ordered product's stock by the ordered quantity.	Gives the new SUPPLIER_ORDER entity an actual effect on inventory, rather than just recording that an order happened.
Supplier order item quantity must be greater than 0, and unit cost cannot be negative.	Matches the same rule already applied to TRANSACTION_ITEM quantity, applied to the new SUPPLIER_ORDER_ITEM table.
A sale event's end date cannot be earlier than its start date.	Basic date-range sanity check for the new SALE_EVENT entity.
Discount end date cannot be earlier than its start date, and discount amount cannot be negative.	Same reasoning, for the existing DISCOUNT entity, which had no such rule in the original constraints list.
A reward tier's minimum points must be unique within its program, and higher tiers require more points than lower ones.	Without this, two tiers could require the same point total, or a higher-named tier could require fewer points than a lower one.
A customer's tier must match a tier whose minimum points is at or below the customer's current point total.	Keeps the cached Tier_id on CUSTOMER consistent with their actual point total; enforced in the application layer since it depends on a calculation, not a single column.
A tax rate's effective end date, if any, cannot be earlier than its effective start date.	Same date-range check as sale events and discounts, applied to the new TAX_RATE entity.
Only managers can edit inventory, prices, and discounts.	Restates the description's rule directly as a constraint; enforced in the application layer, since the database cannot check who is logged in.